mail related data like accounts, aliases, quota information, etc. It's not mandatory to use PostgreSQL. You can use anything else that's supported. I chose it because of my previous experience with it and my future plans.

Consider a typical case of when you are trying to send a mail using a mail client like Thunderbird. After you have supplied the account information to Thunderbird, when you hit the *Send* button, it will contact Postfix to send the mail. Before sending a mail, you must be authenticated. This authentication is usually accomplished using SASL (Simple Authentication and Security Layer).

Postfix has the ability to use Dovecot as an authentication backend, so that it can verify the username and password. Once that is done, Postfix will let you send the messages. With each message, it will check whom it is addressed to. There is a provision in Postfix to reduce the possibility of spam by allowing authenticated users to send mails only from their own addresses or any of the aliases they own.

Now, Postfix will forward the mail to Amavisd via SMTP on a separate port. Amavisd then calls ClamAV (or any other virus scanner) and SpamAssassin to verify that the mail doesn't contain a virus or any spam. A mail server sending spam to other servers can be blacklisted and this has serious repercussions—getting a server whitelisted again is a difficult task.

On completing verification by Amavisd, it will send back the mail to Postfix on a separate port using the SMTP protocol, or quarantine/discard the mail (depending on the configuration).

If Amavisd forwards the mail back to Postfix, it is queued for delivery. Postfix looks up the domain name's MX (Mail Exchange) DNS records. The mail exchange records simply specify the servers that would accept mail for the domain, and their priorities (used for redundancy). In case the MX record is not found for the target domain, the A record is used.

Postfix will try each server listed in the MX records one by one to deliver mail to them. If all the servers refuse to accept mail, the mail just lies in the queue and Postfix keeps trying periodically till the queue expires (by default, the queue expiry time is five days—for undelivered messages, it can be changed).

Coming to Dovecot, it is a powerful IMAP/POP server with loads of features and is designed for performance.

In traditional UNIX mail systems, there used to be just one MTA (something like Postfix), which would take care of delivering mail to a Maildir or MBOX file on the local system. Users would use their shell accounts to read mail using the command line clients.

The traditional model is quite inconvenient and you need to give shell accounts to each user, which is not a good idea. Dovecot can be combined with Postfix to set up Virtual Domain email hosting, by which a particular server can

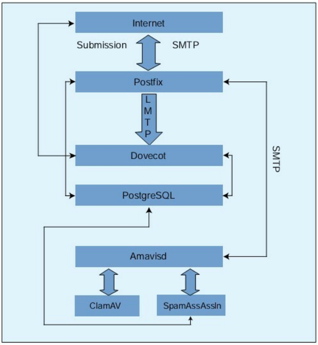

Figure 1: Mail-Flow

accept and store/retrieve emails for any number of domains (well, not really unlimited—it depends on the hardware).

Configuring the PostgreSQL database

As said earlier, PostgreSQL is used for storing accounts, aliases and related data. There are a couple of tools like *postfixadmin* and the like, which let you manage your mail server.

These control panels or tools propose their own database schema (structure). I did try them out but I wasn't satisfied with the features they offered. So I set out to use my own schema and write something like a control panel later.

Table: *domains*

Our domains table has three fields —ID, name and active. ID is the domain number, used as reference in other tables. Name is the domain's name. Active is a Boolean stating whether the server should accept mail for the domain or not.

```
create table domains (
id bigserial primary key,
name character varying (50),
active boolean,
unique(name));
```

View: *active_domains*

We created a domains table earlier, but for performance reasons and to simplify queries in configuration files, use a view that lists only active domains:

```
create view active_domains as select id, name from domains
```